

RGuide: Fast and Accurate Congestion Control Guided via Explicit Rate Control in Data Center Networks

Wentao Wang¹, Jiangping Han¹, *Member, IEEE*, Kaiping Xue¹, *Senior Member, IEEE*, Jiayu Yang¹,
Jian Li¹, *Senior Member, IEEE*, Qibin Sun¹, *Fellow, IEEE*, and Jun Lu¹

Abstract—Congestion control (CC) is crucial in data center networks (DCNs), providing high throughput and low latency transmission services for diverse applications. Existing CC schemes typically rely on iterative rate adjustment at ends, and suffer from performance issues such as slow convergence, throughput fluctuations, and fairness defects. Explicit rate control (ERC) promises to address these challenges by allowing switches to directly allocate rates for each flow, freeing senders from heuristic detection of available bandwidth. However, current ERC-based schemes employ inefficient feedback control to regulate the allocated rates, resulting in sub-optimal performance. In this paper, we propose RGuide, a fast and accurate CC scheme based on ERC. RGuide can calculate accurate fair share rates in real-time at switches with the consideration of low latency, and utilize the rate to guide host adjustments instead of the need for end-to-end iteration processes. We meticulously design the ERC trigger conditions, enabling switches to recognize the different congestion states of flows and rectify flows that deviate from the fair share rate at sub-RTT timescales. We conduct actual testbed experiments and extensive simulations to evaluate RGuide comprehensively. The results demonstrate the significant advantages of RGuide in terms of convergence speed, throughput stability, and fairness.

Index Terms—Data center networks, congestion control, explicit rate control.

I. INTRODUCTION

DATA center networks (DCNs) have become critical infrastructures serving various applications such as parallel computing [1], [2], [3], big-data analytics [4], and distributed

machine learning training [5]. These applications generate large amounts of communication data between servers [6], [7], causing congestion in the networks. Meanwhile, these applications demand high transmission performance; for example, tasks like Spark shuffle and machine learning reduction require substantial throughput [8], [9], while resource disaggregation necessitates microsecond-scale network latency [10]. Congestion control (CC) provides a method for each traffic to compete for available bandwidth resources while maintaining low latency, thus playing an important role in DCNs.

Over the past decade, many efforts have been devoted to enhancing the performance of CC in DCNs. Existing CC schemes [11], [12], [13], [14], [15], [16] typically employ iteration-based rate adjustment algorithms like additive increase multiplicative decrease (AIMD). These algorithms, coupled with end-to-end congestion signals such as explicit congestion notification (ECN) [17] and delay, ultimately complete bandwidth allocation among flows. Schemes in recent years, benefiting from the increased processing capacity of switches in DCNs [18], lean toward leveraging in-network telemetry (INT) [19] to obtain more accurate link information for rate adjustments [20], [21], [22], [23].

However, the iteration-based rate adjustment of senders introduces several performance issues. Firstly, the inherent convergence time of these algorithms prevents flows from promptly adjusting to optimal rates as congestion conditions change. Senders typically require multiple round-trip times (RTTs) to incrementally adjust their rate to match the fair rate at the bottleneck. Particularly, as link bandwidth grows, more flows scale at the bandwidth-delay product (BDP) level, i.e., one RTT at that bandwidth is sufficient to complete the transmission, which poses challenges in achieving convergence before the completion of their transmission [23], [24], [25], [26]. Secondly, due to the lack of effective detection of available bandwidth, existing CCs are compelled to increase the rate continuously based on acknowledgments or timers, even after achieving fair sharing [27]. The continuous adjustments result in a sawtooth-like dynamic equilibrium in the steady state, leading to throughput fluctuations [28], [29]. Moreover, commonly used congestion signals like ECN and delay exhibit a fairness defect. In multi-hop scenarios, flows are marked with greater probability or measured with higher delays. These algorithms misinterpret this as higher network congestion, resulting in undesired rate reduction and unfairness issue [22].

Received 21 August 2024; revised 3 April 2025; accepted 30 May 2025. Date of publication 9 June 2025; date of current version 7 October 2025. This work is supported in part by the National Natural Science Foundation of China under Grant No. 62302472, and Youth Innovation Promotion Association of the Chinese Academy of Sciences (CAS) under Grant No. Y202093. The associate editor coordinating the review of this article and approving it for publication was G. Leduc. (*Corresponding authors: Jiangping Han; Kaiping Xue.*)

Wentao Wang is with the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230027, Anhui, China.

Jiangping Han, Kaiping Xue, Jian Li, and Qibin Sun are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, Anhui, China (e-mail: jphan@ustc.edu.cn; kpxue@ustc.edu.cn).

Jiayu Yang is with Pengcheng Laboratory, Shenzhen 518055, Guangdong, China.

Jun Lu is with the School of Cyber Science and Technology and the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230027, Anhui, China.

Digital Object Identifier 10.1109/TNSM.2025.3577951

The root cause of these issues lies in the lack of accurate rate allocation information, which forces senders to ambiguously probe fair rates through heuristic algorithms. Therefore, explicit rate control (ERC) is expected to break through the limitations faced by existing mechanisms, which is driven by switches explicitly providing rate adjustment information such as rate change values or direct bottleneck fair rates to guide the actions of senders.

Despite the performance enhancement provided by existing ERC-based schemes [30], [31], [32], [33], they have not fully harnessed the potential of ERC. For explicit information that is crucial to ERC, these schemes typically use feedback control [34] at switches for calculation and regulation. This method introduces convergence time and inaccuracy in the explicit information itself, leading to sub-optimal performance. Moreover, the effectiveness of feedback algorithms is highly dependent on precise parameter settings, which often require network operators to conduct transform domain analysis for tuning. The complexity of parameter tuning constrains scalability. Therefore, an efficient method for calculating explicit information is needed to fully leverage the benefits of ERC.

In this paper, we propose RGuide, a faster and more accurate CC scheme based on ERC. RGuide employs a ternary state machine at the switch to recognize the congestion state of flows, allowing it to accurately calculate the fair rate meeting the max-min fairness in real-time. The switch sends the fair rate directly to senders, freeing them from heuristic detection. We cautiously design the ERC trigger conditions to allow prompt rectification of flows when the network congestion conditions change, without causing a large message overhead. On the end node, RGuide uses only two variables for rate adjustment based on explicit information, eliminating the need for inefficient iterative algorithms and numerous parameters.

We evaluate RGuide comprehensively through both actual testbed and simulation experiments, comparing it with ERC-based RoCC [33], as well as DCQCN [12] and HPCC [21], representing ECN-based and INT-based CC, respectively. The results show that RGuide achieves the shortest convergence time across various bandwidths and congestion levels, along with the lowest throughput fluctuation in the steady state. In fairness tests, RGuide effectively addresses the fairness defects in the multi-hop scenario and provides excellent max-min fairness. Large-scale simulations show that RGuide reduces the average flow completion time (FCT) by up to 52%, 31%, and 72% relative to DCQCN, HPCC, and RoCC, respectively.

In summary, the contributions of this paper are as follows:

- We introduce an effective scheme for real-time fair rate calculation at switches with the consideration of low latency, further advancing the speed and accuracy of explicit control and addressing the pain points of existing ERC-based schemes.
- We then propose RGuide, a fast and accurate CC scheme via ERC. We meticulously design the ERC trigger conditions, enabling RGuide to accurately identify the congestion state of flows and promptly rectify flows deviating from the fair share at sub-RTT timescales.
- We conduct comprehensive experiments to evaluate the performance of RGuide, demonstrating that RGuide

delivers significant advantages in terms of convergence speed, throughput stability, and fairness.

The rest of this paper is organized as follows. In Section II, we present related work and explain the motivation of RGuide. In Section III, we illustrate and explain the design of RGuide in detail. Section IV presents comprehensive performance evaluations. At last, we conclude this paper in Section V.

II. RELATED WORK AND MOTIVATION

End-to-end congestion control (CC) is necessary for DCNs to deliver high-performance transmission services. Due to the increasing traffic scale and high transmission requirements of applications, CC in DCNs is devoted to providing rapid convergence, stable throughput, low latency, and fairness. In this section, we first introduce related CC schemes, followed by explaining the motivation behind RGuide.

A. Related Work

Existing CCs are mostly based on iterative rate adjustment algorithms combining congestion signals or in-network telemetry (INT) to enable senders to probe the available bandwidth in the network and achieve rate convergence. Besides, some schemes use explicit rate control (ERC), enabling the switch to send feedback information of explicit rate and direct the actions of senders.

ECN and delay. ECN [17] and delay are the most commonly used congestion signals in DCNs, owing to their simplicity. ECN enables switches to mark packets to notify congestion. DCTCP [11] uses ECN for CC in DCNs, which increases the congestion window based on ACKs and decreases the window by the fraction of marked packets. DCQCN [12] is proposed to enhance the performance for large-scale remote direct memory access (RDMA) deployments, which builds upon DCTCP. DCQCN marks packets at the switch based on double thresholds to notify congestion and utilizes a timer and a byte counter to increase the rate of senders. Delay is another congestion signal widely used in DCNs. TIMELY [13] is based on RTT thresholds to guide adjustments and perform gradient-based increases/decreases. Swift [14] compares the RTT and target delay to adjust the rate.

More accurate in-network information. In recent years, more CCs have tended to use INT to obtain more precise link load information for rate adjustment. HPCC [21] uses INT to get link loads such as queue size and accumulated traffic to adjust the send window. PowerTCP [20] introduces the concept of network power and reacts to both the current state of networks and the trend, avoiding slow reactions and overreactions. Poseidon [22] utilizes INT and adjusts the rate based on the congestion level of the bottleneck hop to address the unfairness issue. Bolt [23] switches assist in rate adjustment by sending customized messages directly to sources, which contain information like link queue lengths, transmission rates, etc.

Explicit rate control. ERC allows switches to explicitly provide a rate change value or a direct fair rate and frees senders from iterative tuning. ERC is initially widely used

TABLE I
COMPARISON OF EXISTING CONGESTION CONTROL CATEGORIES AND STRATEGIES

Congestion Control	Category	Rate Adjustment Strategy	Switch Action
DCTCP [11] DCQCN [12]	ECN-based	Acknowledgement-based, mark probability-based MD Timer and byte counter-based AI, mark probability-based MD	Mark packets based on a threshold Mark packets based on thresholds
TIMELY [13] Swift [14]	Delay-based	RTT and RTT gradient-based AIMD RTT-based AIMD	None None
HPCC [21] PowerTCP [20] Poseidon [22] Bolt [23]	INT-based	Acknowledgement-based AI and inflight packet-based MD Inflight packet and its variation-based adjustment Bottleneck hop delay-based MIMD Acknowledgement and path queue-based AIAD	Insert link load into packets per hop Insert link load into packets per hop Insert maximum hop delay into packets Send customized messages to sender
CAPC [36] ERICA [37] XCP [30] RCP [31] D ³ [32] RoCC [33]	Explicit control	Modifying the explicit rate field Modifying the explicit rate field Explicit window adjustment value Explicit fair rate Explicit rate allocation Explicit fair rate	Iteratively calculate fair rate Iteratively calculate fair rate Iteratively calculate feedback value Iteratively calculate fair rate Iteratively allocate rate Iteratively calculate fair rate
RGuide	Explicit control	Explicit fair rate of bottleneck hop	Calculate fair rate in real-time

in asynchronous transfer mode (ATM) networks, where the source can generate special probe packets called resource management (RM) cells. An RM cell contains an explicit rate field, which can be modified by switches on the path and fed back to the source through the receiver, enabling explicit bandwidth allocation [35]. Representative work such as CAPC [36] uses the input rate to evaluate the link utilization, based on which the fair rate is iteratively computed and added to the RM cells. ERICA [37] employs a similar strategy and uses bandwidth equalization to guarantee a lower bound. In addition, ERC is also widely used in high-speed networks. XCP [30] adds a new congestion header for attaching per-flow congestion information, and the switches on the path calculate the window adjustment value based on the window and delay information, and modify this field to control the congestion windows of the sources. RCP [31] calculates the fair rate at the switch based on the queue length and input rate and feeds it back to the sender through the added rate field in packets. D³ [32] explicitly allocates bandwidth at the switch based on the rate demand of flows, which is calculated from the remaining flow size and deadline. To avoid under-utilization and reduce queue, D³ still uses an iterative control based on observed utilization and queue to adjust the available bandwidth. In DCNs, RoCC [33] uses a PI controller [34] to calculate the fair rate at the switch, which stabilizes the queue length around a reference value. RoCC periodically sends the fair rate directly to the sender corresponding to each flow to guide rate adjustment via specific ICMP messages.

A summary and comparison of existing congestion control categories and strategies is shown in Table I.

B. Motivation

Iterative rate adjustment faces performance limitations. Existing CCs typically employ iterative algorithms such as AIMD and MIMD to establish rules for inter-flow bandwidth competition, which ultimately converge the rate to fair sharing. However, the iterative nature of these algorithms introduces convergence time, particularly problematic

in high-speed networks where FCT is typically very short. A recent study indicates that almost 80% of remote procedure calls (RPCs) in DCNs can complete their transmission within 4 RTTs at 100 Gbps [23], while the existing iteration-based CCs typically converge in tens of RTTs. Consequently, these CCs struggle to achieve the optimal bandwidth allocation before the completion of flows [38], [39]. Moreover, due to the lack of explicit awareness of available bandwidth, sender-initiated bandwidth probing often increases the rate to get more utilization. These schemes increase periodically or based on ACKs, even when fair sharing has been reached. In the steady state, these algorithms are dynamically balanced under the control of congestion signals. Although the average rate reaches the fair rate on a certain time scale, they constantly perturb the balance causing large throughput fluctuations. Additionally, iterative adjustment may suffer from fairness defects and fail to converge to max-min fairness. As commonly used congestion signals, ECN and delay only reflect the congestion of the whole path and lack the granularity of the hop level. In multi-hop scenarios, flows are marked more or measured with higher delays and become victim flows. These victim flows compete for lower throughput than other flows that traverse a single congested hop. This unfairness can also happen in INT-based schemes, caused by inappropriate algorithms that react to any hop congestion [21].

We believe that the root cause of these performance impairments is the lack of precise and direct control information, which forces the flows to compete ambiguously for bandwidth resources through iterative algorithms.

Existing ERC-based CCs have not fully exploited the benefits. ERC avoids senders' ambiguous detection for bottleneck fair rates. Thus, ERC is more suitable for high-speed networks and is expected to break through the limitations faced by existing CCs. The state-of-the-art ERC-based CC like RoCC [33] offers better improvement in convergence speed and bandwidth stability. However, these schemes calculate fair rates still based on iterative algorithms at switches, which decrease the allocated bandwidth when queues grow and

increase the allocated bandwidth when links are underutilized. The switch cannot provide immediate and accurate explicit control for senders as congestion changes. These algorithms' convergence time and error prevent existing ERC-based schemes from fully exploiting the advantages of explicit control in terms of bandwidth convergence and stability. Additionally, these algorithms utilize the parameters such as queue length and link utilization, which are involved in all flows. Flows that do not contribute to congestion also participate in these algorithms' regulation, thus failing to avoid multi-hop unfairness. Our experimental results reveal this phenomenon (Section IV).

Our design goals. In light of these issues, our objective is to develop a CC scheme that is free from iterative algorithms. Our design goals involve (i) employing explicit control-based rate adjustment at senders to avoid the performance drawbacks associated with rate iteration, and (ii) redesigning the fair rate calculation algorithm with requirements for real-time and accuracy.

III. RGUIDE DESIGN

We propose RGuide, a novel ERC-based scheme for fast convergence, high throughput stability, and fairness in DCNs. In this section, we first elaborate on the framework of RGuide, then show the design details of switches and hosts, and finally analyze the parameter settings.

A. Framework of RGuide

The framework of RGuide is illustrated in Fig. 1. RGuide operates as a switch-driven CC scheme, which enables the switch to calculate the current fair rate in real-time and assign it to senders via ERC. Each host only uses the received ERC to adjust its sending rate without any other operations. RGuide comprises three key designs at switches: a flow table, a fair rate calculator, and a ternary state machine. Firstly, RGuide utilizes the flow table to maintain congestion information for active flows including source rate, congestion state, etc. Secondly, RGuide combines flow table information and the current queue length to perform real-time fair rate calculations with a focus on low latency. Next, RGuide builds the ternary state machine, which traces the congestion state of flows based on the rate variations and determines the conditions for triggering ERC. When a flow deviates from the fair rate, RGuide enables the switch to send a specific message packet containing the fair rate directly to the corresponding sender.

On the sender, RGuide needs to modify the packet header to add a new field for the source to append the current transmission rate. Fig. 2 shows an example of the RGuide source rate field in RDMA over Converged Ethernet version 2 (RoCE v2), which can be extended from the RDMA Base Transport Header (BTH). Every packet of each flow fills the field with the current rate so that switches on the path can quickly update the congestion information and determine the congestion state of flows. RGuide avoids the use of complex iterative algorithms; the sender simply obtains the fair rate from the switch for fast and accurate rate adjustment.

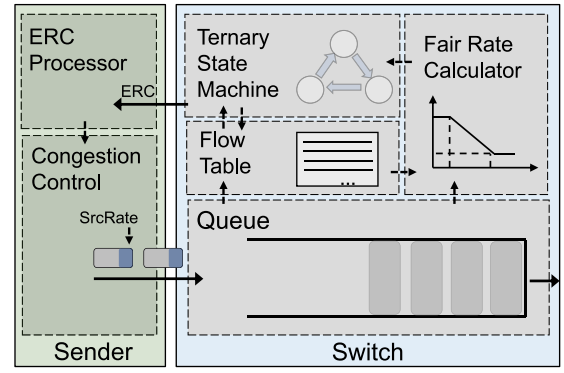


Fig. 1. The framework of RGuide.

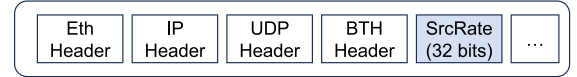


Fig. 2. The modified header of RGuide in RoCE v2. We use Kbps as the unit, and 32 bits is sufficient to support the maximum available transmission rate.

B. Switch Algorithm

1) *Flow Table*: RGuide maintains a flow table for each switch egress port to record the congestion information of outgoing flows and determine the congestion level. A flow table entry includes the identification of a flow (*flowId*), the source rate (*srcRt*), the fair rate of the last performed ERC (*lstRt*), the corresponding timestamp (*lstTs*), and *flag*, where *flowId* is obtained by hashing quintuples of packets, *srcRt* is updated by reading the added source rate field of packets, and *flag* is used to label the congestion state of flows.

We define three options for *flag*. In the case where the bottleneck of a flow *i* resides at the switch (hereafter referred to as a bottlenecked flow), *flag_i* is set to BN, indicating that flow *i* contributes to congestion at this port. Conversely, if flow *i* is determined to be a non-bottlenecked flow, i.e., constrained elsewhere in the network and not contributing to congestion at this switch, *flag_i* is set to non-BN. If flow *i* is in an undetermined state, *flag_i* is set to TBD (to be determined).

For each table entry, we use Kbps as the unit of *srcRt* and *lstRt*. 32-bit is sufficient to support the maximum transfer rate. For the timestamp, we can set it to 32-bit data. For *flag*, it only needs two bits. Therefore, the overhead per table entry is no more than 13 bytes.

2) *Calculation of the Fair Rate*: RGuide calculates the fair share rate at switches, which is used to guide rate adjustment for flows. The existing schemes calculate the fair rate by iterative algorithms [31], [33], [40], [41], which require a certain number of iterations. Our goal is to achieve faster CC, so RGuide redesigns the algorithm to provide real-time fair rates at the switch.

Fair rate that meets max-min fairness. The rate allocation on switches needs to satisfy max-min fairness. It ensures that the flows that go through a link receive the same rate [40], i.e., the fair rate R_f , unless their rates are constrained by other factors, such as end hosts or other switches, to a smaller value. Specifically, we call a flow that does not reach the fair

rate a non-bottlenecked flow at the switch. Otherwise, it is called a bottlenecked flow at the switch. Let R_i denote the constrained rate of flow i by other factors within the network, and S denote the flow set going through the target link on the switch. For flow i , the switch can eventually allocate a bandwidth of $\min\{R_f, R_i\}$. For the link that is fully utilized, the fair rate should satisfy:

$$BW = \sum_{i \in S} \min\{R_f, R_i\}, \quad (1)$$

where BW is the link bandwidth.

Bottlenecked flows cause congestion at the switch, tending to gain more rate with increased available bandwidth. Thus each bottlenecked flow is allocated bandwidth of R_f . Non-bottlenecked flows are limited by other nodes, which are manifested by remaining below the fair rate R_f even if more available bandwidth exists. Therefore, we can get:

$$BW = \sum_{i \in S_n} R_i + N_B R_f, \quad (2)$$

where N_B is the number of bottlenecked flows, and S_n is the set of non-bottlenecked flows. Then, the ideal fair rate assigned to the bottlenecked flows at the congestion point is calculated by:

$$R_f = \frac{BW - \sum_{i \in S_n} R_i}{N_B}. \quad (3)$$

Algorithm for fair rate calculation. Based on Eq. (3) as well as the consideration for low latency, the way we designed to calculate R_f in our deployment is as follows:

$$R_f = \max\left\{\frac{\alpha BW}{N_a}, \frac{\alpha BW - \sum_{i \in S_n} srcRt_i}{N_b}\right\}, \quad (4)$$

where N_a is the number of active flows (an active flow is defined as a flow whose table entry has been updated during one RTT) and N_b is the number of bottlenecked flows and undetermined flows ($flag$ is BN or TBD). N_a , N_b and $srcRt_i$ are maintained by the flow table. We use $\frac{\alpha BW}{N_a}$ to ensure a lower bound in the rate allocation for robustness, while the other term is designed based on Eq. (3).

We design a bandwidth headroom factor α for low latency requirements. Because when the bandwidth achieves a reasonable allocation, the queue length may be at a high level. Therefore, we set α to control the bandwidth that can be allocated. The factor α is calculated based on the instantaneous queue length $qLen$. As shown in Fig. 3, we use two queue length thresholds: $maxTh$ and $minTh$. When $qLen$ is below $minTh$, α is set to 1, indicating that all bandwidth is allocated to flows. When $qLen$ is higher than $maxTh$, α is set to the minimum α (α_{min}) to devote most of the bandwidth to draining the queue. In other cases, α decreases linearly as the instantaneous queue length $qLen$ grows, i.e.,

$$\alpha = 1 + \frac{qLen - minTh}{maxTh - minTh}(\alpha_{min} - 1). \quad (5)$$

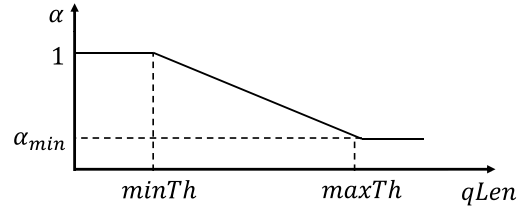


Fig. 3. The bandwidth headroom factor α setting logic.

3) *Ternary State Machine and ERC*: The main challenge of real-time fair-rate calculation is identifying bottlenecked and non-bottlenecked flows without global information. To address this challenge, RGuide employs a ternary state machine to detect the per-flow congestion state based on rate variation at the switch. We meticulously design state transfer conditions and provide triggers for ERC.

For newly arriving flow i , one ERC is provided directly to adjust it to the appropriate rate quickly; then $flag_i$ is set to TBD; $lstRt_i$ is set to the fair share rate R_f ; and $lstTs_i$ is updated to the current time $curTs$.

For the flow i that already has a table entry in the flow table, to trigger ERC the time condition needs to be met first:

$$curTs - lstTs_i \geq timeTh, \quad (6)$$

where $timeTh$ is a time threshold, i.e., the switch triggers ERC only at most once for a flow in a $timeTh$ time. The time threshold is to ensure that the ERC is delivered to the sender, avoiding multiple repetitive ERCs.

If the time condition i.e., Eq. (6) is satisfied, the rate condition is selected according to $flag_i$. If $flag_i = TBD$, then the switch further determines whether the following inequality is satisfied:

$$|srcRt_i - lstRt_i| < rateTh, \quad (7)$$

where $rateTh$ is a rate threshold. If Eq. (7) is satisfied, it means that the flow i adopts the last ERC and it is a bottlenecked flow of the switch, then set $flag_i = BN$. Otherwise, the flow i does not adopt the ERC, indicating that its rate is limited at other nodes, and then it is a non-bottlenecked flow, so set $flag_i = non-BN$.

Then, if $flag_i = BN$, the following rate condition must be met to trigger ERC:

$$|srcRt_i - R_f| \geq rateTh, \quad (8)$$

i.e., for bottlenecked flow, when its actual rate differs from the ideal fair rate, it is adjusted directly to the appropriate value through ERC. Then, $flag_i$ is set to TBD; $lstRt_i$ is set to R_f ; and $lstTs_i$ is updated as $curTs$.

If $flag_i = non-BN$, the rate condition is:

$$srcRt_i - R_f \geq rateTh. \quad (9)$$

For a non-bottlenecked flow, the switch is not responsible for increasing its rate. It is only controlled when its rate exceeds the fair rate, indicating that its bottleneck changes to the switch, and then $flag_i$ is set to BN. Fig. 4 shows the ternary state machine of $flag$ and corresponding transfer conditions.

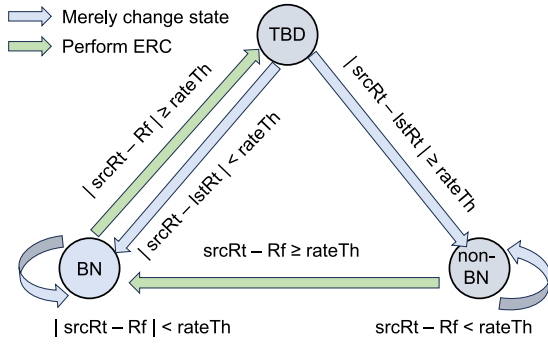


Fig. 4. The ternary state machine of *flag* and corresponding transfer condition. At most one state transfer is performed within a *timeTh*.

A flow may experience different state transfers at different switches and is typically labeled as BN at only one switch, which is the one with the smallest fair rate on the path.

If the rate condition Eq. (8) is used for a non-bottlenecked flow, because its rate is limited by other hops, this switch will keep trying to send ERC, which confuses the sender. Therefore, we use *flag* to indicate the congestion state of flows and use different rate conditions. Because of the complex traffic in DCNs, the bottleneck of a flow may change multiple times, so RGuide sets the *flag* to TBD after each ERC to verify whether the flow is still the bottlenecked flow or not.

The entire deployment of the switch algorithm can be shown by Algorithm 1.

C. Host Algorithm

RGuide offloads most of the operations in CC to the network, so the end nodes simply perform very few operations. RGuide introduces no changes at the receiver, and the sender only needs to use two variables to achieve fast and precise rate adjustment without iterative processes.

To avoid rapid queue buildup on the path, RGuide designs the initial send rate to be one-tenth of the line rate. Before packets are transmitted, the sender fills in the added fields shown in Fig. 2 with the current send rate. The RGuide sender adjusts the rate based on the ERC message packets received. The deployment of the sender algorithm can be shown by Algorithm 2. When the sender receives an ERC, it gets the explicit fair rate. If no other ERC is received within the previous *timeTh* time or the fair rate of the ERC is less than the last one, the sender adopts this ERC and adjusts its rate to the fair rate to match the transmission capacity of the bottleneck. Adopting a lower rate is because the flow rate should be set according to the bottleneck link. According to the above ERC trigger conditions, other switches except the bottleneck switch will treat it as a non-bottlenecked flow and stop rectifying it, thus realizing a unique match between the sender and the bottleneck.

D. Parameters of RGuide

Rather than relying on complex control algorithms, RGuide uses only intuitive yet effective logical judgments to achieve CC. Therefore, the deployment of RGuide does not have strict requirements on the parameters. RGuide mainly has several

Algorithm 1: The Switch Algorithm

input: The packet *pkt*.

```

1 id ← getFlowId(pkt);
2 curTs ← getCurrentTimestamp();
3 if id = NULL then
4   id ← InitializeTableEntry(fTbl);
5   fTbl[id].flag ← TBD;
6   frRt ← getFairRate();
7   performERC(id, frRt);
  // lstTs and lstRt are updated in
  // this function.
8 return;
9 fTbl[id].srcRt ← getSrcRate(pkt);
10 if curTs - fTbl[id].lstTs ≥ timeTh then
11   if fTbl[id].flag = TBD then
12     if abs(fTbl[id].srcRt - fTbl[id].lstRt) < rateTh then
13       fTbl[id].flag ← BN;
14     else
15       fTbl[id].flag ← non-BN;
16   frRt ← getFairRate();
17   if fTbl[id].flag = BN then
18     if abs(fTbl[id].srcRt - frRt) ≥ rateTh then
19       performERC(id, frRt);
20       fTbl[id].flag ← TBD;
21       return;
22   else
23     if fTbl[id].srcRt - frRt ≥ rateTh then
24       performERC(id, frRt);
25       fTbl[id].flag ← BN;
26       return;

```

Algorithm 2: The Sender Algorithm

```

1 Function handleERC(ERC)
2   frRt ← getFairRate(ERC);
3   if frRt < lstFrRt or curTs - lstTs ≥ timeTh then
4     setTransmitRate(frRt);
5     lstFrRt ← frRt;
6     lstTs ← curTs;

```

threshold parameters: *maxTh*, *minTh*, *timeTh* and *rateTh*, along with a minimum bandwidth headroom factor α_{min} . The two queue thresholds, *maxTh* and *minTh*, are used to signify the level of queue buildup, which can be set according to existing queue-based schemes [12], [42]. *timeTh* is primarily used to ensure that the previous ERC is already delivered and working each time the switch makes a congestion status determination. In our deployment, we set it to double the base inter-server delay. In practical deployments, senders may not set the rate consistently according to the explicit fair rate due to setting accuracy. Therefore, instead of using strict equations for the

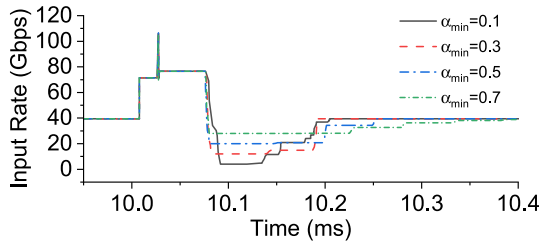


Fig. 5. Comparison of draining queues with different bandwidth headroom factors. The burst starts at 10 ms.

rate conditions, we use the rate threshold $rateTh$ to allow for error. In our testbed, we set $rateTh$ to 10 Kbps.

In RGuide, α_{min} indicates the minimum bandwidth percentage that can be allocated to active flows when the queue is high. In our deployment, we typically set it to 0.3. We show through an experiment that different parameters have little effect on RGuide, where 10 bursts perturb a 40 Gbps bottleneck. The total bandwidth allocation with different parameters is shown in Fig. 5. Intuitively, lower factors drain more efficiently but also introduce more perturbation to the allocated bandwidth. Drain times are typically in the hundred-microsecond range, which has little effect on rate convergence.

IV. PERFORMANCE EVALUATION

In this section, we evaluate the performance of RGuide by a real-world testbed based on DPDK [43] and OMNet++ [44] simulations. We mainly verify RGuide's substantial advantages in terms of convergence speed, throughput stability, delay, and fairness. Additionally, we use large-scale simulations to show that RGuide helps reduce flow completion time. We mainly compare RGuide with DCQCN [12], HPCC [21], which use ECN and INT for CC, respectively, and RoCC [33], a CC based on ERC. DCNs are typically configured consistently and deployed with a single CC algorithm by network operators. We therefore focus on evaluating the performance of RGuide itself and do not address inter-protocol coexistence.

For RGuide, the two queue thresholds, $minTh$ and $maxTh$, are set to 20 KB and 100 KB, respectively, α_{min} is configured to 0.3, $timeTh$ is set to about twice the base RTT for each scenario, and $rateTh$ is set to 10 Kbps. The parameters for other schemes are set based on their respective papers.

A. DPDK Evaluation

We first verify RGuide's performance through testbed experiments. Our testbed contains four nodes, one of which acts as a software switch connecting the other three nodes. Each node is equipped with a Core i5-11500 processor, 64 GB DDR4 RAMs, and Intel E810 10 GbE NICs. We implement user-state stacks based on DPDK and deploy RGuide and DCQCN. In the test scenario, we set up two senders to transmit traffic to one receiver, one of the flows is sent first, and the second flow is transmitted as a burst with a duration of 1 s. We observe the variation of the throughput.

The obtained results, illustrated in Fig. 6, demonstrate RGuide's remarkable convergence speed and throughput stability. As shown in Fig. 6(a), when the burst of traffic begins,

DCQCN spends about 400 ms bringing the two flows to their fair share. When entering the steady state, the throughput fluctuates around the fair rate under the control of rate self-incrementation and congestion notification packets (CNPs). In contrast, RGuide immediately directs the sender to adjust rates via ERC to achieve the fastest rate convergence in theory once the burst of traffic begins. In the steady state, RGuide maintains stable throughput without the need for sender-initiated rate probing, thus achieving extremely stable throughput as shown in Fig. 6(b).

Throughout the experiment, RGuide generates only four control message packets, i.e., the explicit control for flows at the time of their injection and the control for flow 1 when flow 2 starts and ends. This precise control enables efficient tuning of the source rate. Fig. 7 shows the number of control messages each flow receives in each scheme. According to our statistics, each DCQCN flow receives approximately 750 CNPs on average during the injection of flow 2, i.e., to accomplish rate convergence and steady-state dynamic balance, DCQCN needs to provide one CNP to each flow every 1.3 ms. The use of coarse-grained congestion signals and sender-driven available bandwidth probing results in notable message overhead and frequent rate adjustments.

B. Micro-Benchmarks

We then conduct multiple micro-benchmarks to evaluate the performance of each scheme in terms of convergence speed, stability, delay, and fairness.

1) *Convergence Speed*: We use a many-to-one topology to evaluate the convergence speed of each scheme, where multiple senders transmit data to a single receiver through a bottleneck switch. An intuitive result is shown in Fig. 8. In this scenario, each flow starts sending at 10 ms intervals and then stops sending in reverse order. The bandwidth is set to 40 Gbps and the delay of each link is set to 1.5 μ s. The switch buffer size is 600 KB. DCQCN cannot achieve bandwidth allocation in the network that changes many times in a short period. HPCC uses more accurate congestion signals and offers great improvements in convergence. However, there is still a range of throughput fluctuations after convergence. RoCC and RGuide provide the best convergence so that the flows can quickly get their fair share of bandwidth. Because RoCC uses the PI controller to calculate the fair rate, the throughput allocation can be unstable at the beginning and end of a flow, as shown in Fig. 8(c). RGuide achieves excellent performance both in terms of fast convergence and stability.

Response to short bursts. We inject a short flow (100 KB, about 3 bandwidth-delay products) that shares the bottleneck with a long flow to evaluate the resilience of the schemes to the impact of the burst. As shown in Fig. 9, DCQCN's throughput is the most impacted. After the burst injection, the long flow is slowed down via ECN. When the short flow ends, the DCQCN's sender slowly ramps up the rate, leading to a long period of link under-utilization. HPCC exhibits a higher tolerance for the burst. Although the throughput of RoCC is affected very little, in practice the switch does not inhibit the rate of the sender because the queue length is not up to the

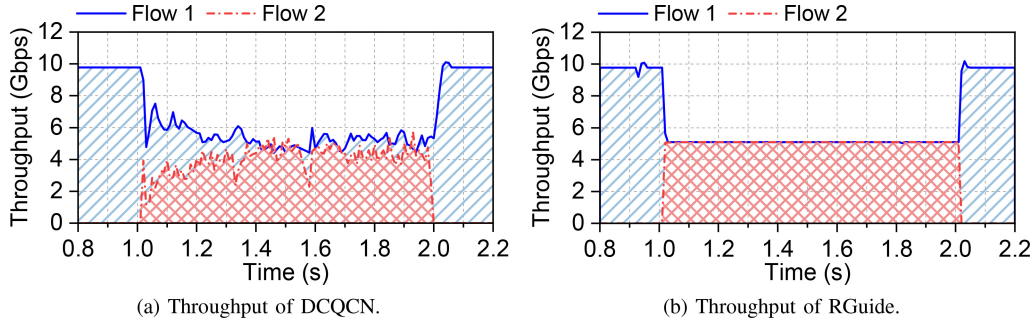


Fig. 6. Measured throughput of DCQCN and RGuide in testbed versus time.

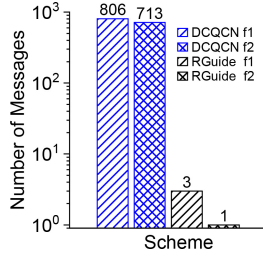


Fig. 7. Number of control messages received by each flow.

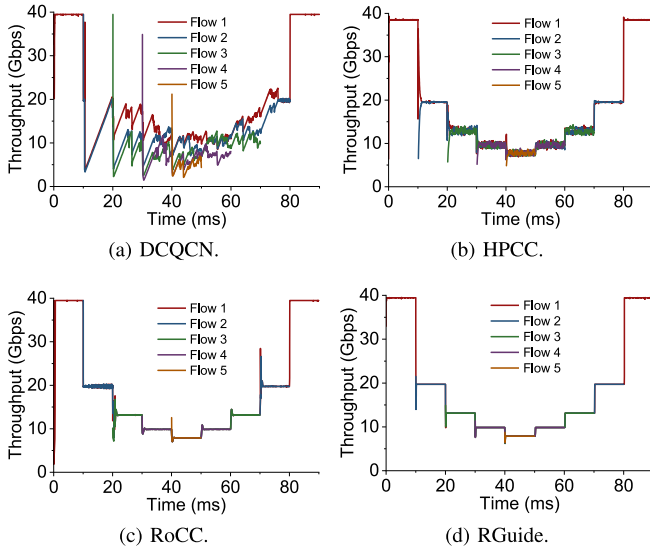


Fig. 8. Bandwidth sharing in convergence test.

set reference value, which leads to long queue buildup after the short flow is completed. RGuide achieves higher burst tolerance than HPCC. With fast explicit bandwidth allocation, senders can converge quickly to avoid high queues and under-utilization.

Convergence of long concurrent flows. We further evaluate the convergence ability of the schemes at different network parameters. We set the number of senders to 2. One sender transmits first, and after occupying the bandwidth of the bottleneck link, the second sender starts sending. We vary the bandwidth and measure convergence time, which is defined as the earliest time point that ensures that the subsequent throughput is within $\pm 10\%$ or $\pm 500\text{Mbps}$ of the fair rate. The result is shown in Fig. 10. DCQCN's convergence time increases as

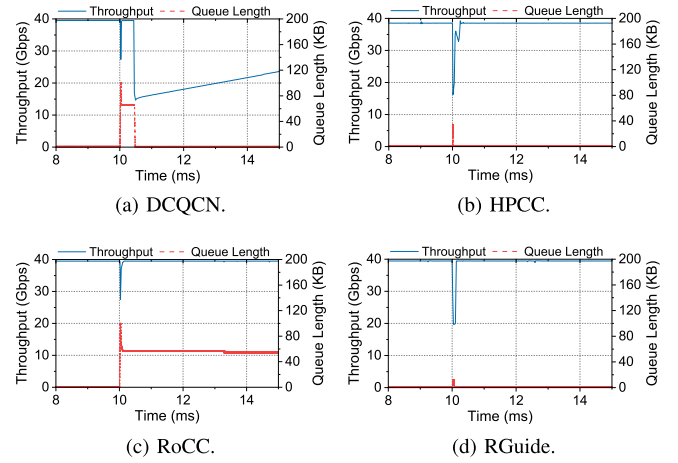


Fig. 9. Impact of short burst flows on the throughput of long flows and queue length.

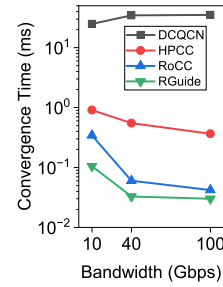


Fig. 10. Convergence time versus bandwidths.

the bandwidth increases. HPCC, RoCC, and RGuide benefit from faster message delivery for shorter convergence times as bandwidth increases. HPCC uses INT to get detailed link information and greatly reduces the convergence time to about 10 RTTs. RoCC uses iteration-free rate adjustment and shorter signal feedback paths, and thus achieves shorter convergence time. RGuide further uses real-time fair-rate computation and achieves the fastest convergence, providing up to 99.9%, 94%, and 45.3% reduction in convergence time compared to the three schemes, respectively.

To evaluate convergence under different degrees of incast, we change the number of senders and make them start transmitting simultaneously. We exponentially increase the number of concurrent flows from 2 to 128. The bandwidth

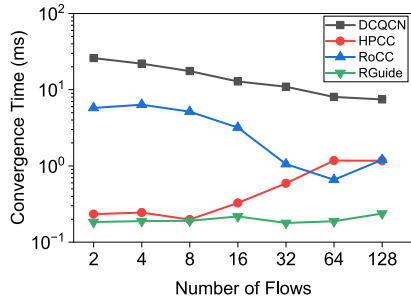


Fig. 11. Convergence time with different number of concurrent flows.

is set to 100 Gbps. As illustrated in Fig. 11, RGuide consistently exhibits the shortest convergence time even in highly congested scenarios. Conversely, DCQCN delivers the convergence time at 10 ms level. As the number of concurrent flows rises, the fair bandwidth diminishes, resulting in shorter climb times for the flows and a decreasing trend in DCQCN's convergence time. Through this experiment, we show that RoCC struggles to maintain faster convergence when flows are sent simultaneously due to the instability of the switch iteration algorithm. Although HPCC shows excellent convergence speed in scenarios with a small number of concurrent flows, its convergence time extends as incast intensifies.

2) *Throughput Stability and Delay*: We also compute the standard deviation of the rate and measure the queue length distribution in the steady state to evaluate stability and latency. The experiment scenario remains a many-to-one topology with 40 Gbps bandwidth and 600 KB switch buffer size. The link delay of each hop is set to 1.5 μ s. We set up 2, 32, and 64 concurrent flows to correspond to light, medium, and heavy incast scenarios. Each experiment measures the throughput and queue length after the flows are injected for one second to guarantee that all flows reach the steady state.

Throughput stability. The standard deviation of the rates of each scheme with different degrees of incast scenarios is shown in Table II, which characterizes the throughput stability of the schemes. The results show that DCQCN has the poorest stability, attributed to its utilization of timer-based increases and ECN-based decreases in the steady state. Compared to DCQCN, HPCC achieves fine-grained rate tuning, minimizing perturbations in the steady state. RoCC further improves throughput stability through ERC. RGuide excels in maintaining stability. In the steady state, RGuide does not enable the switch and the senders to perform any action, thus guaranteeing that the rate remains stable at the fair share. As shown in Table II, when under light and medium incast, the flows do not even adjust in the steady state. In heavy congestion, due to bursts of queue buildup, the switch will occasionally reduce the allocated bandwidth to drain the queue, causing a slight degree of rate adjustment.

Queue delay. The queue length distribution for each scheme is shown in Fig. 12, which demonstrates that RGuide also has advantages in reducing delay. DCQCN causes large queue buildup and jitter in the steady state. Its queue jitter deteriorates further as concurrent flows increase. RoCC calculates fair rates at switches based on queue buildup, varies the fair rates

TABLE II
RATE STANDARD DEVIATION (MB/S) AT STEADY STATE WITH VARYING DEGREES OF INCAST

Scheme	Light incast	Medium incast	Heavy incast
DCQCN	1879.20	337.38	303.74
HPCC	94.79	108.92	28.72
RoCC	27.87	36.86	9.37
RGuide	0	0	0.32

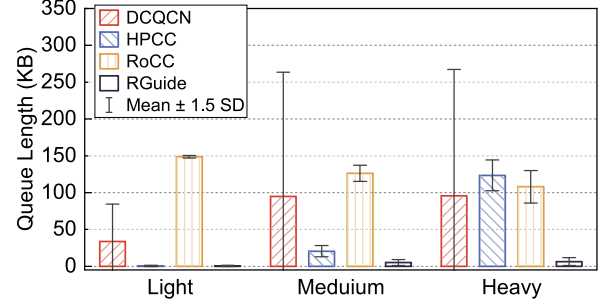


Fig. 12. Queue length at steady state with different degrees of incast.

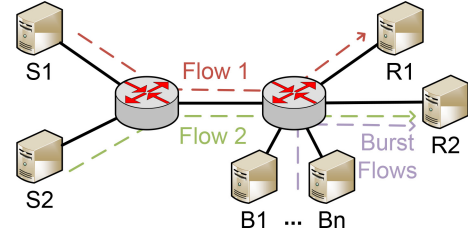


Fig. 13. Topology of the fairness test.

by comparing the queue length with the reference value, and finally achieves convergence based on feedback control. Thus, in the steady state, the queue stabilizes around the reference value of 150 KB. Although RoCC improves the queue jitter, the higher reference value increases the queuing delay of flows. HPCC is effective in reducing queue lengths when congestion levels are low. However, as a window-based CC, it cannot provide finer rate control. As the level of congestion increases, the queuing delay and jitter of HPCC gradually increase. With heavy congestion, HPCC loses its low-latency advantage. RGuide uses an adaptive bandwidth headroom factor when allocating bandwidth. With significant queue buildup, RGuide allocates more bandwidth for draining the queue. Upon the queue falling back, RGuide then reallocates bandwidth to flows, guaranteeing low latency while achieving good bandwidth allocation. In addition, RGuide's queuing delay is almost unaffected by increasing congestion levels.

3) *Fairness*: We evaluate whether maximum-minimum fairness can be achieved and the victim flow issue in multiple scenarios. The topology we use is shown in Fig. 13. The link bandwidth is 40 Gbps and the delay is 1 μ s. In this scenario, flow 1 and flow 2 share one bottleneck link, and flow 2 and burst flows share another bottleneck.

Max-min fairness. We constantly increase the number of burst flows at 50 ms intervals. Flow 2 shares the bottleneck with the burst flows, and thus its throughput continues

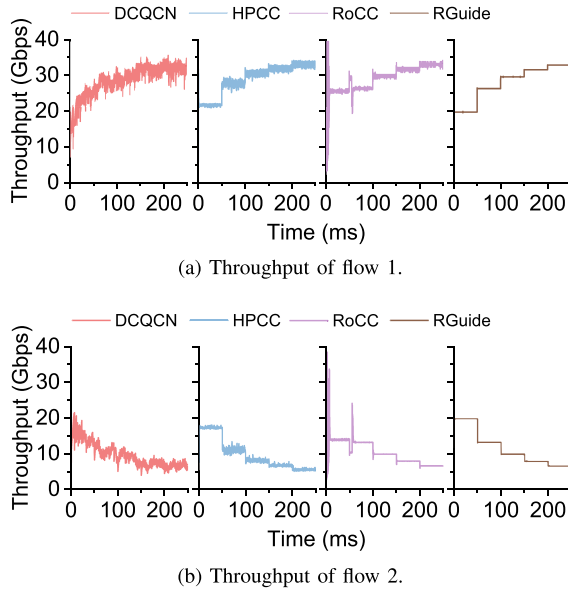


Fig. 14. Variation of throughput as the number of burst flows increases.

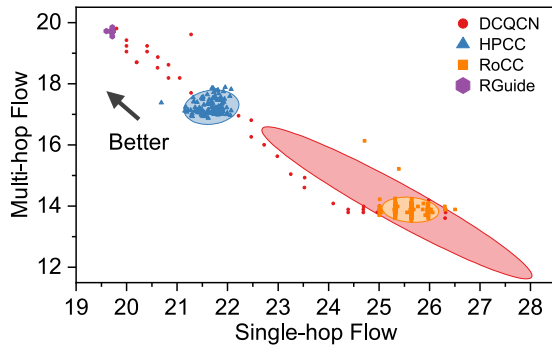


Fig. 15. Throughput distribution of the Multi-hop and single-hop flows.

to decrease. Therefore, flow 1 should receive more bandwidth according to max-min fairness and exhibit a step-wise increase.

The bandwidth allocation for each flow is shown in Fig. 14. We can conclude that DCQCN provides a coarse-grained bandwidth reallocation when network congestion changes. HPCC and RoCC achieve relatively fine-grained bandwidth allocation in comparison, and the throughput shows an approximate step-wise increase. HPCC takes less time to reach the steady state, but the throughput of senders also fluctuates. In addition, the throughput of flow 2 is slightly lower than the fair bandwidth. Although RoCC achieves max-min fairness in the steady state, the throughput shows large fluctuations at the beginning of the congestion change.

RGuide shows a significant advantage in achieving bandwidth allocation that satisfies max-min fairness. In this scenario, RGuide can recognize flow 2 as a non-bottlenecked flow at the first hop. As the burst flows increase and flow 2 reduces its rate, the switch at the first hop allocates more of the remaining available bandwidth via ERC to flow 1. As shown in Fig. 14, the flows in the RGuide exhibit the desired step-wise throughput variation.

Multi-hop fairness. We then verify whether each scheme can maintain fairness as flows traverse multiple congestion points. We first send flow 1 and flow 2, and when they reach a steady state, inject a burst flow shown in Fig. 13. Each flow should be allocated half of the bandwidth. However, flow 2 of the scheme with fairness defects will slow down more and become a victim flow. Fig. 15 presents the throughput distribution after the burst flow injection. We can conclude that only RGuide succeeds in providing a fair share of bandwidth for each flow. In contrast, DCQCN and RoCC fail to achieve fairness, with flow 2 getting only half the throughput of other flows in the steady state. For DCQCN, because the packets of flow 2 are marked at both congestion points, its sender reduces the rate more frequently, causing it to compete for lower bandwidth. Similarly, RoCC's switch algorithm uses the queue length to adjust the fair rate so that non-bottlenecked flows are also involved. For HPCC, because INT is a congestion signal with hop-level congestion accuracy, it can theoretically solve the unfairness in this scenario [22]. However, as shown in Fig. 15, HPCC only mitigates the unfairness, and the throughput of the victim flow is still lower than the single-hop flow. This is because HPCC reacts to congestion on any hop, while it only needs to adjust its rate based on the congestion level at the bottleneck.

RGuide achieves excellent fairness and each flow's throughput is almost the same. An RGuide switch can identify bottlenecked and non-bottlenecked flows and attempts to regulate only bottlenecked flows, avoiding undesired suppression of flows that do not contribute to congestion. End nodes are guided by the ERC of the bottleneck hop, which avoids regular congestion signals such as ECN and delay being affected by the cumulative congestion. Additionally, benefiting from the design of ERC, RGuide also realizes a smaller throughput jitter as shown in Fig. 15.

C. Large-Scale Simulations

We further use large-scale simulations to evaluate the flow completion time (FCT) of each scheme, which records the time taken for a flow from the start to the completion of its transmission. The topology we use is a two-level fat-tree [45]. Each core switch connects to the edge switches by 100 Gbps, and edge switches have 40 Gbps links connecting to multiple servers. Each switch queue has a capacity of 1 MB. The delay for each hop is 1.5 μ s. We use *WebSearch* traffic [11] as the network load. The flow generation follows a Poisson distribution. Each node randomly selects the destination end.

We measure the average FCT and the 90th percentile FCT, and the results are shown in Fig. 16. Fig. 16(a) and Fig. 16(b) show the average FCT of each size flow. DCQCN and RoCC have longer queues at the switches, resulting in a longer FCT for short flows. RGuide takes the queue length into account when allocating bandwidth. When the queue length is long, RGuide uses α to drain the queue. Thus, RGuide can reduce the FCT of delay-sensitive short flows. For DCQCN and RoCC, RGuide can reduce 28% to 52% and 53% to 72% of FCT, respectively. In addition, the fast convergence of RGuide allows the rate of senders to be quickly adjusted to the right

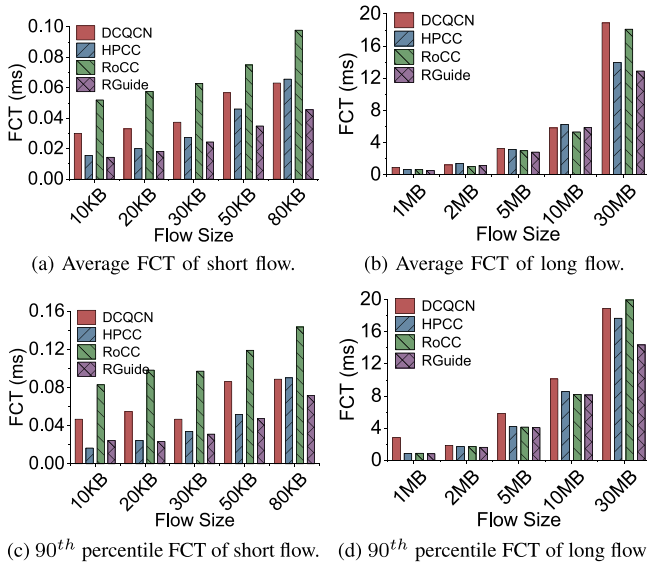


Fig. 16. FCT of DCQCN, HPCC, RoCC, and RGuide.

value to fully utilize the bandwidth. Therefore, for bandwidth-sensitive long flows, RGuide also helps to reduce their FCT. DCQCN converges slowly, resulting in some long flows not fully utilizing the bandwidth and increasing FCT. For long flows greater than 1 MB, RGuide reduces FCT by 32% to 45% compared to DCQCN. As the size of the flow increases, the advantage of HPCC over FCT diminishes. For medium and long flows, RGuide reduces FCT by up to 31% relative to HPCC.

Fig. 16(c) and Fig. 16(d) show the 90th percentile FCT of each size flow. We can see that RGuide achieves lower tail FCT in most cases. RoCC's short flows have high tail FCT due to higher queues, while DCQCN's long flows have high tail FCT due to underutilized bandwidth. HPCC has the advantage of reducing the latency of short flows by slightly reducing bandwidth utilization, which leads to performance impairments for long flows. RGuide achieves lower tail FCT. For short flows, RGuide reduces the tail FCT by up to 57%, 20%, and 76% compared to DCQCN, HPCC, and RoCC, respectively. For long flows, the corresponding performance improvements are 70%, 19%, and 28%, respectively.

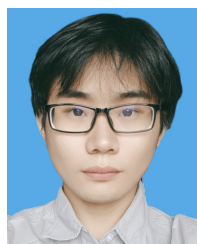
V. CONCLUSION

In this paper, we revealed the performance impairment issues caused by the inherent defects of existing iteration-based CCs. To break through the limitations faced by existing schemes, we proposed RGuide, a fast and accurate CC scheme based on ERC. RGuide can determine the congestion state of flows, accurately calculate the fair share rate in real-time at switches, and provide direct guidance to senders for rate adjustment. Experimental results demonstrate that RGuide greatly accelerates rate convergence, achieves almost zero throughput jitter and low delay in the steady state, provides excellent max-min fairness, and effectively reduces flow completion time.

REFERENCES

- [1] Y. Yu et al., "DryadLINQ: A system for general-purpose distributed data-parallel computing using a high-level language," in *Proc. 8th USENIX Conf. Oper. Syst. Design Implement. (OSDI)*, 2008, pp. 1–14.
- [2] T.-W. Huang, D.-L. Lin, C.-X. Lin, and Y. Lin, "Taskflow: A lightweight parallel and heterogeneous task graph computing system," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1303–1320, Jun. 2022.
- [3] Y. Dong et al., "Faster and scalable MPI applications launching," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 2, pp. 264–279, Feb. 2024.
- [4] J. Camacho, K. Wasielewska, R. Bro, and D. Kotz, "Interpretable feature learning in multivariate big data analysis for network monitoring," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2926–2943, Jun. 2024.
- [5] X. Zhao and C. Wu, "Large-scale machine learning cluster scheduling via multi-agent graph reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4962–4974, Dec. 2022.
- [6] S. Xie, G. Hu, C. Xing, and Y. Liu, "Online elephant flow prediction for load balancing in programmable switch-based DCN," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 745–758, Feb. 2024.
- [7] Y. Liu, J. Han, K. Xue, J. Li, Q. Sun, and J. Lu, "DECC: Achieving low latency in data Center networks with deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4313–4324, Dec. 2023.
- [8] M. Zaharia et al., "Resilient distributed Datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Syst. Design Implementation (NSDI)*, 2012, pp. 15–28.
- [9] C. Lao et al., "ATP: In-network aggregation for multi-tenant learning," in *Proc. 18th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2021, pp. 741–761.
- [10] P. X. Gao et al., "Network requirements for resource disaggregation," in *Proc. 12th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2016, pp. 249–264.
- [11] M. Alizadeh et al., "Data center TCP (DCTCP)," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2010, pp. 63–74.
- [12] Y. Zhu et al., "Congestion control for large-scale RDMA deployments," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2015, pp. 523–536.
- [13] R. Mittal et al., "TIMELY: RTT-based congestion control for the datacenter," in *Proc. 2015 ACM Conf. Special Interest Group Data Commun. (SIGCOMM)*, 2015, pp. 537–550.
- [14] G. Kumar et al., "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2020, pp. 514–528.
- [15] M. Long, J. Han, W. Wang, J. Yang, and K. Xue, "LSCC: Link-segmented congestion control for RDMA in cross-datacenter networks," in *Proc. 32nd IEEE/ACM Int. Symp. Qual. Service (IWQoS)*, 2024, pp. 1–10.
- [16] J. Han, K. Xue, W. Wang, R. Li, Q. Sun, and J. Lu, "RateMP: Optimizing bandwidth Utilization with high burst tolerance in data Center networks," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2024, pp. 1361–1370.
- [17] A. Ford, C. Raiciu, M. Handley, and O. Bonaventure, "The addition of explicit congestion notification (ECN) to IP," Internet Eng. Task Force, RFC 3168, 2001. Accessed: Jun. 2025. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc311>
- [18] "In-band network telemetry in Broadcom tomahawk 3." Accessed: Jun. 2025. [Online]. Available: <https://www.broadcom.com/company/news/product-releases/2372840>
- [19] K. Yang et al., "SketchINT: Empowering INT with TowerSketch for per-flow per-switch measurement," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 11, pp. 2876–2894, Nov. 2023.
- [20] V. Addanki, O. Michel, and S. Schmid, "PowerTCP: Pushing the performance limits of datacenter networks," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 51–70.
- [21] Y. Li et al., "HPCC: High precision congestion control," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2019, pp. 44–58.
- [22] W. Wang et al., "Poseidon: Efficient, robust, and practical datacenter CC via deployable INT," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 255–274.
- [23] S. Arslan, Y. Li, G. Kumar, and N. Dukkupati, "Bolt: Sub-RTT congestion control for ultra-low latency," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 219–236.
- [24] B. Montazeri, Y. Li, M. Alizadeh, and J. Ousterhout, "Homa: A receiver-driven low-latency transport protocol using network priorities," in *Proc. ACM Conf. Special Interest Group Data Commun. (SIGCOMM)*, 2018, pp. 221–235.

- [25] S. Hu et al., "Aeolus: A building block for proactive transport in datacenters," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Archit., Protoc. Comput. Commun. (SIGCOMM)*, 2020, pp. 422–434.
- [26] R. Joshi et al., "Masking corruption packet losses in datacenter networks with link-local retransmission," in *Proc. Conf. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2023, pp. 288–304.
- [27] S. Liu, A. Ghalayini, M. Alizadeh, B. Prabhakar, M. Rosenblum, and A. Sivaraman, "Breaking the transience-equilibrium nexus: A new approach to datacenter packet transport," in *Proc. 18th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2021, pp. 47–63.
- [28] M. Alizadeh, A. Javanmard, and B. Prabhakar, "Analysis of DCTCP: Stability, convergence, and fairness," *SIGMETRICS Perform. Eval. Rev.*, vol. 39, no. 1, pp. 73–84, 2011.
- [29] Y. Zhu, M. Ghobadi, V. Misra, and J. Padhye, "ECN or delay: Lessons learnt from analysis of DCQCN and TIMELY," in *Proc. 12th Int. Conf. Emerg. Netw. Exp. Technol. (CoNEXT)*, 2016, pp. 313–327.
- [30] D. Katabi, M. Handley, and C. Rohrs, "Congestion control for high bandwidth-delay product networks," *SIGCOMM Comput. Commun. Rev.*, vol. 32, no. 4, pp. 89–102, Aug. 2002.
- [31] N. Dukkipati and N. McKeown, "Why flow-completion time is the right metric for congestion control," *SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 1, pp. 59–62, 2006.
- [32] C. Wilson, H. Ballani, T. Karagiannis, and A. Rowtron, "Better never than late: Meeting deadlines in datacenter networks," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2011, pp. 50–61.
- [33] D. Menikkumbura, P. Taheri, E. Vanini, S. Fahmy, P. Eugster, and T. Edsall, "Congestion control for datacenter networks: A control-theoretic approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1682–1696, May 2023.
- [34] R. Pan, L. Breslau, B. Prabhakar, and S. Shenker, "Approximate fairness through differential dropping," *SIGCOMM Comput. Commun. Rev.*, vol. 33, no. 2, pp. 23–39, 2003.
- [35] S. Muddu, F. Chiu, C. Tryfonas, and V. Kumar, "Max-min rate control algorithm for available bit rate service in ATM networks," in *Proc. Int. Conf. Commun. (ICC)*, vol. 1, 1996, pp. 412–418.
- [36] A. Arulambalam, X. Chen, and N. Ansari, "Allocating fair rates for available bit rate service in ATM networks," *IEEE Commun. Mag.*, vol. 34, no. 11, pp. 92–100, Nov. 1996.
- [37] S. Kalyanaraman, R. Jain, S. Fahmy, R. Goyal, and B. Vandalore, "The ERICA switch algorithm for ABR traffic management in ATM networks," *IEEE/ACM Trans. Netw.*, vol. 8, no. 1, pp. 87–98, Feb. 2000.
- [38] P. Goyal, P. Shah, N. K. Sharma, M. Alizadeh, and T. E. Anderson, "Backpressure flow control," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 779–805.
- [39] J. Zhang, X. Zhong, Z. Wan, Y. Tian, T. Pan, and T. Huang, "RCC: Enabling receiver-driven RDMA congestion control with congestion divide-and-conquer in datacenter networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 103–117, Feb. 2023.
- [40] Z. Yu, J. Wu, V. Braverman, I. Stoica, and X. Jin, "Twenty years after: Hierarchical core-stateless fair queueing," in *Proc. 18th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2021, pp. 29–45.
- [41] X. Zhong, J. Zhang, Y. Zhang, Z. Guan, and Z. Wan, "PACC: Proactive and accurate congestion feedback for RDMA congestion control," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2022, pp. 2228–2237.
- [42] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Trans. Netw.*, vol. 1, no. 4, pp. 397–413, Aug. 1993.
- [43] "DPDK." Accessed: Jun. 2025. [Online]. Available: <https://www.dpdk.org/about/>
- [44] "OMNeT++ discrete event simulator." Accessed: Jun. 2025. [Online]. Available: <https://omnetpp.org/>
- [45] C. E. Leiserson, "Fat-trees: Universal networks for hardware-efficient supercomputing," *IEEE Trans. Comput.*, vol. C-34, no. 10, pp. 892–901, Oct. 1985.



Wentao Wang received the bachelor's degree from the Department of Electronic Engineering and Information Science, USTC in July 2022, where he is currently pursuing the master's degree. His research interests include transmission optimization and data center networks.



Jiangping Han (Member, IEEE) received the bachelor's and Doctoral degrees from the Department of Electronic Engineering and Information Science, USTC in 2016 and 2021, respectively. From November 2019 to October 2021, She was a Visiting Scholar with the School of Computing, Informatics, and Decision Systems Engineering, Arizona State University. She was a Postdoctoral Researcher with the School of Cyber Science and Technology, USTC from 2021 to 2023, where she is currently an Associate Researcher with the School of Cyber Science and Technology. Her research interests include future Internet architecture design and transmission optimization.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC) in 2003, and the Doctoral degree from the Department of Electronic Engineering and Information Science, USTC in 2007. From May 2012 to May 2013, he was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering, University of Florida. He is currently a Professor with the School of Cyber Science and Technology, USTC. His research interests include next-generation Internet architecture design, transmission optimization, and network security. He serves on the editorial board of several journals, including the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and the IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. He is an IET Fellow.



Jiayu Yang received the bachelor's and Doctoral degrees from the School of Cyber Security, University of Science and Technology of China in 2019 and 2024, respectively. She is currently an Assistant Researcher with Pengcheng Laboratory, Shenzhen, China. Her research interests include future Internet architecture design, transmission optimization, and network security.



Jian Li (Senior Member, IEEE) received the bachelor's degree from the Department of Electronics and Information Engineering, Anhui University in 2015, and the Doctoral degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC) in 2020. From November 2019 to November 2020, he was a Visiting Scholar with the Department of Electronic and Computer Engineering, University of Florida. From December 2020 to December 2022, he was a Postdoctoral Researcher with the School of Cyber Science and Technology, USTC, where he is currently an Associate Researcher with the School of Cyber Science and Technology. His research interests include quantum networking, wireless networks, and next-generation Internet. He serves as an Editor for *China Communications*.



Qibin Sun (Fellow, IEEE) received the Ph.D. degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China in 1997, where he is currently a Professor with the School of Cyber Science and Technology. He has published more than 120 papers in international journals and conferences. His research interests include multimedia security, network intelligence, and security.



Jun Lu received the bachelor's degree from Southeast University in 1985, and the master's degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China in 1988, where he is currently a Professor with the School of Cyber Science and Technology. He is also the President of Jiaxing University. His research interests include theoretical research and system development in the field of integrated electronic information systems, network, and information security. He is an Academician of the Chinese Academy of Engineering.